# EWE

## Easy Weaponry Enchanter

### System Design Document

Unreal Engine C++ Plugin | v1.0

| | |
|---|---|
| **Document Version** | v1.0 |
| **Engine Version** | Unreal Engine 5.x |
| **Design Completeness** | ~90% (Editor tooling TBD) |
| **Required Plugins** | GameplayAbilities, EnhancedInput |

Apache License

Copyright 2026 Jae Hyeong Lee.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

# 1. Overview

EWE is an Unreal C++ plugin designed to assign event-driven ability combinations to weapons — answering the question: 'What happens when the player clicks the mouse?'

It is not a skill crafting tool, but an Event → Condition → Function compositor that operates as an abstraction layer on top of GAS (Gameplay Ability System).

## 1.1 Design Goals

- Abstraction of GAS-based event-function structure
- Blueprint-friendly enchant system
- Bidirectional authoring pipeline: in-game and editor
- Extensible to AI and world objects
- Full multiplayer support

## 1.2 Core Composition Structure

| Stage | Role | Example |
|---|---|---|
| Trigger | When does it fire? | LMB click, On Hit, On Kill |
| Condition | Under what conditions? | HP below 50%, count limit, GameplayTag filter |
| Event Type | How does it fire? | One-shot, Continuous, Sustained, Charged |
| Function | What does it do? | Spawn, Eject, Rotate, Teleport, etc. |
| Effect Result | What is the outcome? | Damage, object spawn, status effect |

# 2. System Architecture

## 2.1 GAS Integration

| Component | Owner | Notes |
|---|---|---|
| AbilitySystemComponent (ASC) | Character | GA re-registered on weapon swap |
| AttributeSet | Character | Reset to weapon defaults on swap |
| GameplayAbility (GA) | Defined in weapon DataAsset → registered to Character | 1 Enchant = 1 GA |
| GameplayEffect (GE) | Character | Cooldowns, buffs, debuffs |
| AbilityTask (AT) | Inside GA | 1 Function = 1 AbilityTask |

## 2.2 Weapon Swap Lifecycle

- Processing begins upon receiving a weapon swap Confirm from the server
- All active GAs (not yet End/Cancelled) are forcibly Cancelled
- Character AttributeSet is reset to the new weapon's default values
  - Current HP: retained (clamped to new Max HP if exceeded)
  - Character Position and velocity: excluded from reset
  - Active GEs (including cooldowns): retained as-is
- New weapon's GAs are registered to the ASC
- Enchant slot editing is only allowed when the weapon is not equipped

## 2.3 Network Authority

- All enchants and GAs execute only after server Confirm (no client-side prediction)
- No client feedback before server Confirm
- Spawned independent objects: Server is Owner; damage/effect calculation uses the spawning player as reference
- Weather Change: Server request → broadcast to all clients
- Teleport: Server validates destination → sends corrected position to client

## 2.4 Attribute Schema Policy

- Each weapon may define a variable Attribute schema, but cannot reference Attributes absent from the Character's AttributeSet
- Referencing a non-existent Attribute: console Warning printed, entry silently ignored
- Attribute input is free-form (no dropdown restriction)

# 3. Enchant System

## 3.1 Enchant Structure (UEnchantDataAsset)

Each enchant is defined as a single DataAsset and implemented internally as a GA containing multiple AbilityTasks.

| Field | Type | Description |
|---|---|---|
| Trigger | FEnchantTrigger | Fire condition (TriggerType Enum + InputAction) |
| Conditions[] | TArray<FEnchantCondition> | Multi-condition array, AND-combined |
| EventType | EEnchantEventType (Enum) | One-shot / Continuous / Sustained / Charged |
| CooldownGE | TSubclassOf<UGameplayEffect> | Per-enchant cooldown GE (optional) |
| Functions[] | TArray<FEnchantFunction> | AbilityTask bundle to execute |
| SlotIndex | int32 | Slot index within the weapon |

## 3.2 Slot Policy

- Maximum enchant count per weapon is set by the creator in the DataAsset
- Execution order is fixed to slot assignment order
- Each enchant operates independently; fires simultaneously by default
- Sequential execution only when a 'PrecedingEnchantEnd' condition is set
- Sustained and Charged enchants follow the same concurrency rules
- Duplicate enchants on the same weapon are allowed — each runs as a fully independent GE instance (Aggregate)

## 3.3 GA Lifecycle

| State | Definition | Counts as 'Ended' for sequencing |
|---|---|---|
| EndAbility | Normal completion (duration expired, etc.) | Yes |
| CancelAbility | Forced cancellation (weapon swap, interrupt) | No |
| Active | Neither End nor Cancel has been called | Quick slot swap restricted |

## 3.4 AbilityTask Execution Policy

- One GA may contain multiple AbilityTasks
- All Tasks execute simultaneously unless a condition dictates otherwise
- GA ends when all AbilityTasks have completed
- Cooldown GE cancellation is treated identically to EndAbility

# 4. Trigger System

## 4.1 Trigger Types

| Trigger Type | Fire Condition | Notes |
|---|---|---|
| OnSpawn | When the actor/object is spawned | |
| OnAttack | When a hit is registered on another actor | Not on input, on hit confirmation |
| OnHit | When hit condition is met (regardless of damage) | Fires once per hit instance |
| OnKill | When a target is killed | |
| OnDeath | When the caster dies | |
| OnInput | On specific key input | Enhanced Input / IMC-based |
| OnCombo | State condition AND input condition | e.g. HP < 50% + RMB |
| OnCollision | When a projectile collides with an actor/object | Can deliver GE to the collision target |

## 4.2 Input System

- Enhanced Input system; key bindings defined via InputMappingContext (IMC)
- Default fire keys: Left Mouse Button (LMB) and Right Mouse Button (RMB)
- Additional key bindings can be extended by the developer via IMC

## 4.3 OnHit Multi-Fire Restriction

- Enchants with no cooldown: fires once per hit instance
- Enchants with a cooldown GE active: blocked on both client and server
- No client feedback when blocked

# 5. Condition System

## 5.1 Condition Types

| Condition Type | Description | Parameters |
|---|---|---|
| CountLimit | Maximum fire count | MaxCount: int32 |
| ValueCondition | Attribute value threshold check | AttributeTag, Operator, Threshold |
| TagFilter | Target GameplayTag filter | RequiredTags, BlockedTags |
| SetEffect | Set bonus condition | RequiredSetTags |
| Synergy | Synergy condition | SynergyTags |
| PrecedingEnchantEnd | Fire after preceding enchant ends (EndAbility only) | PrecedingSlotIndex: int32 |

## 5.2 Target Type Filter

- Target actors are filtered using GameplayTags
- RequiredTags: tags the target must possess
- BlockedTags: tags that prevent the enchant from firing on that target
- Example tag taxonomy: Character.Player, Character.AI, Object.Destructible, Environment

# 6. Function System

## 6.1 Common Context Parameters

All Functions share the following context structure.

| Parameter | Type | Description |
|---|---|---|
| Who | AActor* | Reference actor (caster) |
| When | EFunctionTiming | Execution timing |
| Where | EPositionBase | Position basis (Self / Target / World) |
| Target | FGameplayAbilityTargetDataHandle | Target handle |
| How | EFunctionMethod | Execution method |
| Context | FEnchantContext | Parent weapon and caster reference |

## 6.2 Creation (Spawn)

| Parameter | Type | Description |
|---|---|---|
| PositionBase | EPositionBase | Self or Target as origin |
| Offset | FVector | Offset from origin |
| SpawnTiming | ESpawnTiming | Immediate or delayed |
| SpawnActorClass | TSubclassOf<AActor> | Actor class to spawn |
| Lifetime | float | Lifespan (0 = condition-only, -1 = permanent) |
| DestroyConditions[] | TArray<FEnchantCondition> | Destroy condition list (no list = no auto-destroy) |
| OnDestroyEnchant | FEnchantEventRef | Enchant event to fire on destruction (optional) |
| FacingDirection | EFacingDirection | Spawn facing direction |
| SpawnEffect | UParticleSystem* | Spawn visual effect |
| bUsePooling | bool | Enable object pooling |
| bReplicates | bool | Enable network replication |

- If no DestroyConditions are set, the object will not be destroyed except by Lifetime expiry
- On destruction, OnDestroyEnchant fires — enabling chained enchant sequences
  - Example: projectile destruction spawns shrapnel; object explosion spreads a status effect

## 6.3 Ejection / Fire

| Parameter | Type | Description |
|---|---|---|
| ProjectileClass | TSubclassOf<AProjectile> | Projectile class |
| Speed | float | Launch speed |
| bHoming | bool | Enable homing |
| OnCollisionGE | TSubclassOf<UGameplayEffect> | GE applied to collision target on impact |
| CollisionEnchantEvent | FEnchantEventRef | Enchant event to chain-fire on collision (optional) |
| FireCount | int32 | Number of projectiles to fire |
| bPenetrate | bool | Enable penetration |

- Integrates with OnCollision Trigger: OnCollisionGE is applied to the target immediately upon impact
- CollisionEnchantEvent, if set, fires the referenced enchant event on collision

## 6.4 Rotation

| Parameter | Type | Description |
|---|---|---|
| RotationAxis | EAxis | Axis of rotation (X / Y / Z) |
| AngularSpeed | float | Angular velocity (deg/s) |
| bAttachToTarget | bool | Attach rotation to target actor |

## 6.5 Attach (Actor Grouping)

Groups two actors together, maintaining a fixed distance and sharing Forces and GEs.

| Parameter | Type | Description |
|---|---|---|
| Duration | float | Group duration (auto-releases on expiry) |

- First target: the actor the player is looking at when LMB is pressed
- Second target: the actor the player is looking at when LMB is released
- Grouping is applied once both targets are confirmed
- Group auto-releases when Duration expires
- Re-targeting an already-grouped pair immediately releases the group
- Characters and Pawns cannot be targeted
- Grouping effect: fixed distance maintained; Forces and GEs are shared between the two actors
  - Force applied to one actor is equally applied to the other
  - GE applied to one actor is propagated to the other
- If either actor in the group is destroyed, the group is released automatically

## 6.6 Special Functions

### Time Stop

- Effect: freezes Position and velocity of all actors in the session, except the caster
- Attribute changes remain allowed (only position and velocity are frozen)
- All input is blocked (UI interaction allowed, but character-affecting UI actions are blocked)
- Re-activating Time Stop immediately cancels it; duration expiry also cancels it
- If multiple Time Stop requests arrive in the same server tick, all are processed and applied on the next tick

### Freeze

- Effect: freezes Position and velocity of a single target actor
- All input is blocked (UI interaction allowed, but character-affecting UI actions are blocked)
- Active GAs and AbilityTasks are preserved; position/velocity changes from GAs are exempt
- OnHit Trigger-based enchants can still fire while Frozen
- Firing an OnHit enchant while Frozen does not break the Freeze
- A player under Time Stop cannot cast Freeze — unless the Freeze request arrives in the same tick as the Time Stop request, in which case it is processed as lower-priority but still fires
- Functionally equivalent to Stun (no separate Stun status effect)

### Teleport

Destination selection supports three modes, configurable via parameter.

| Parameter | Type | Description |
|---|---|---|
| TeleportMode | ETeleportMode (Enum) | Preview / ActorTarget / Offset |
| OffsetVector | FVector | Offset vector used in Offset mode |

- Preview mode: while LMB is held, a translucent ghost character appears at the nearest valid landing position in view; releasing LMB teleports the player there
- ActorTarget mode: instantly teleports to the position of the actor in view
- Offset mode: teleports by a fixed offset vector relative to the caster
- All modes: server validates the destination (landing validity, collision), confirms, then sends the corrected position to the client

### Possess Object

- Targets the actor currently in view
- Temporarily creates a Pawn component and camera on the target actor, transferring player control to it
- Allowed inputs while possessing: movement, jump, and camera rotation only
- Cannot possess an already-possessed Pawn
- While possessing: all input to the original character is blocked; UI interaction is also blocked
- If the original character is hit while possessing: a red screen border is shown and persists until possession ends
- The player can manually cancel possession at any time

- On possession end: temporary Pawn/camera are removed; view and control return to the original character immediately
- If the possessed actor is destroyed: immediately returns to the original character; no default penalty
  - UI effects on return can be assigned via OnDestroyEnchant in the spawning enchant

## Weather Change

Adjusts world lighting (Directional Light, Sky Atmosphere, etc.) to a specified time of day.

| Parameter | Type | Description |
|---|---|---|
| TargetTimeOfDay | float | Target time of day (0.0–24.0, e.g. 6.0 = 6 AM) |
| TransitionDuration | float | Transition duration in seconds |

- Server receives request → broadcasts updated lighting values to all clients
- Lighting transitions smoothly over the TransitionDuration
- Simultaneous conflicting requests: last request to arrive at the server wins (Last-Write-Wins)

## Status Effect

Implemented via GEs. Each status effect is a discrete GE and is selected in the DataAsset.

| Status Effect | Effect | Notes |
|---|---|---|
| Slow | Reduces movement speed | Attribute-based magnitude |
| Burn | Periodic damage over time | Periodic GE application |
| Silence | Blocks enchant (GA) activation | Independent of cooldown GEs |
| Root | Blocks movement (input still allowed) | Speed clamped to 0; attack input allowed |
| Blind | Obscures vision | Screen effect applied client-side |

- Stun is equivalent to Freeze and is not provided as a separate status effect

## Status Effect Stacking Policy

- All status effects use GE Aggregate — each instance is fully independent, enabling compounding effects
- Different status effects applied simultaneously each maintain independent durations
- Freeze + Slow example: Slow is active but has no effect while Freeze holds speed at 0; Slow becomes effective once Freeze expires
- Silence: OnHit Trigger-based enchants can still fire (same rule as Freeze)

| Common Parameters | Type | Description |
|---|---|---|
| StatusEffectType | EStatusEffectType (Enum) | One of the 5 types above |
| Duration | float | Effect duration |

| Common Parameters | Type | Description |
|---|---|---|
| Magnitude | float | Effect intensity (Slow reduction %, Burn damage, etc.) |
| StatusEffectGE | TSubclassOf<UGameplayEffect> | GE class to apply |

# 7. Cooldown System

## 7.1 Structure

- Cooldowns are implemented as GEs (GameplayEffects)
- Each enchant DataAsset sets its own CooldownGE independently
- Enchants without a CooldownGE have no cooldown

## 7.2 Restriction Policy

- Attempting to fire an enchant while its CooldownGE is active: blocked on both client and server
- No client-side feedback when blocked (silently ignored)
- CooldownGE cancellation is treated the same as EndAbility
- On weapon swap, all GEs including cooldowns are retained (not reset)
- Each enchant has its own cooldown, so player actions are not globally restricted

# 8. Recording System (Recording Book)

## 8.1 Recording Book

- The Recording Book is a special weapon item that carries the Recording GE
- While equipped, pressing LMB applies the Recording GE to the character (begins recording)
- All GAs fired while Recording GE is active are logged in order
- Recording continues across weapon swaps (the book does not need to remain equipped)
- No deduplication — if the same GA fires multiple times, it is recorded each time

## 8.2 Recording End Conditions

- Re-equipping the Recording Book and firing its GA ends recording and creates a Scroll item
- If the enchant slot limit is reached, recording stops automatically and Recording GE is removed

## 8.3 Scroll Item

- One scroll = the full recorded GA bundle
- Multiple scrolls can be held in inventory simultaneously
- Scrolls are applied to weapons via the Enchanter UI
- Applying a scroll does not consume it (can be reused indefinitely)
- If a scroll's GA count exceeds available slots, GAs are assigned sequentially from the front

## 8.4 JSON Export / Import

- Recorded enchant configurations can be exported as a JSON file saved locally
- Other players can import a shared JSON file and apply it to their own weapons
- Enchant configurations are openly viewable by other players
- Tampered JSON files are accepted (open-format design)
- Classes not present on the server are skipped; all other entries are loaded
- Version mismatches are loaded optimistically (compatibility warnings printed to console)

# 9. UI / UX

## 9.1 HUD

- 10 quick slots (slot 0 = Recording Book)
- Slot switching via mouse wheel and number keys
- Quick slot swap is restricted while any GA is still active (not yet End/Cancelled)
- Hovering over a slot displays the enchant list for that weapon

## 9.2 Inventory

- Toggled with the Tab key
- Holds items that can be assigned to HUD quick slots
- Click an item, then press a number key to assign it to a slot

## 9.3 Enchanter UI

- Assign enchants (DataAssets) to weapons and manage slot layout
- Scrolls can be applied to weapons from this UI
- No preview functionality
- Slot editing is only permitted when the weapon is not currently equipped

## 9.4 Possess Object UI

- When the original character is hit during possession: a red border overlay is shown on-screen (applied once, persists until possession ends)
- Existing UI is fully blocked during possession

# 10. Extensibility

## 10.1 Plugin Module Structure

| Module | Role | Dependencies |
|---|---|---|
| EWERuntime | Core runtime (GAs, GEs, DataAssets, Functions, etc.) | GameplayAbilities, EnhancedInput |
| EWEEditor | Editor-only features (DataAsset customization, tooling) | EWERuntime |

- Required plugins: GameplayAbilities, EnhancedInput (no additional dependencies at this time)
- Editor-only code is strictly separated from runtime code via WITH_EDITOR macros

## 10.2 Custom Enchant Extension

- Custom enchant DataAssets can be created by subclassing UEnchantDataAsset
- New Functions can be implemented by conforming to the IEnchantFunction interface
  - Overriding ExecuteFunction() is sufficient to add a new Function
  - Custom Functions operate through the same pipeline as built-in Functions
- Developer-authored C++ Functions are immediately exposed to Blueprint
- Third-party projects can extend EWE by simply adding EWERuntime as a module dependency

## 10.3 AI and World Object Extension

- The enchant system can be applied to any actor that owns an ASC (AI, structures, etc.)
- For AI: extend by binding Triggers to AI events (perception, attack, etc.)
- World objects (traps, installations, etc.) can reuse the same DataAsset structure

# 11. Pending Items

## 11.1 Editor Tooling

⚠ **Editor development plan TBD**

- Undecided whether to provide DataAsset Details panel customization only, or a dedicated editor window (Graph / Tree view)
- Editor UX design will proceed once a direction is confirmed

# 12. Design Completeness

| System Area | Completeness | Outstanding Items |
|---|---|---|
| Network Architecture | 95% | — |
| GAS Integration | 90% | — |
| Enchant Data Structure | 90% | — |
| Trigger System | 95% | — |
| Recording System | 90% | Edge case handling |
| Cooldown System | 90% | — |
| Function Design | 95% | Revisit when editor tooling is integrated |
| UI / UX | 60% | Editor tooling TBD |
| Editor Tooling | 20% | To be designed |